

File permissions in Linux

Project description

Check file and directory details

The following code demonstrates how I used Linux commands to determine the existing permissions set for a specific directory in the file system.

The provided command and its output were utilized to retrieve information about the contents of the projects directory. By employing the "ls" command with the "-la" option, I obtained a comprehensive listing of files, including hidden ones. The resulting output indicates the presence of various items within the directory.

```
researcher2@5d738f0f927b:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 ..
-rw--w---- 1 researcher2 research_team  46 Dec  2 15:27 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec  2 15:27 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Dec  2 15:27 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Dec  2 15:27 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  2 15:27 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  2 15:27 project_t.txt
researcher2@5d738f0f927b:~/projects$
```

Describe the permissions string

The 10-character string can be deconstructed to determine who is authorized to access the file.

- **1st character:** This character is either a d or hyphen (-) and indicates the file type. If it's a d, it's a directory. If it's a hyphen (-), it's a regular file.
- **2nd-4th characters:** These characters indicate the read (r), write (w), and execute (x) permissions for the user. When one of these characters is a hyphen (-) instead, it indicates that this permission is not granted to the user.
- **5th-7th characters:** These characters indicate the read (r), write (w), and execute (x) permissions for the group. When one of these characters is a hyphen (-) instead, it indicates that this permission is not granted for the group.
- **8th-10th characters:** These characters indicate the read (r), write (w), and execute (x) permissions for other. This owner type consists of all other users on the system apart from the user and the group. When one of these characters is a hyphen (-) instead, that indicates that this permission is not granted for other.

Change file permissions

The organization determined that other shouldn't have write access to any of their files.

The following code demonstrates how I used Linux commands to do this.

```
researcher2@5d738f0f927b:~/projects$ chmod o-w project_k.txt
researcher2@5d738f0f927b:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 ..
-rw--w---- 1 researcher2 research_team   46 Dec  2 15:27 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec  2 15:27 drafts
-rw-rw-r-- 1 researcher2 research_team   46 Dec  2 15:27 project_k.txt
-rw-r----- 1 researcher2 research_team   46 Dec  2 15:27 project_m.txt
-rw-rw-r-- 1 researcher2 research_team   46 Dec  2 15:27 project_r.txt
-rw-rw-r-- 1 researcher2 research_team   46 Dec  2 15:27 project_t.txt
researcher2@5d738f0f927b:~/projects$
```

The provided screenshot showcases two command inputs, followed by the output of the second command. The purpose of the "chmod" command is to modify permissions for files and directories. The initial argument signifies which permissions are to be altered, while the second argument specifies the targeted file or directory. In this instance, I withdrew write permissions for the "project_k.txt" file from the "other" category. Subsequently, I executed the "ls -la" command to review the modifications that were implemented.

Change file permissions on a hidden file

The following code demonstrates how I used Linux commands to change the permissions

```
researcher2@3213bbc1d047:~/projects$ chmod u-w,g-w,g+r .project_x.txt
researcher2@3213bbc1d047:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec 20 15:36 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec 20 15:36 ..
-r--r----- 1 researcher2 research_team   46 Dec 20 15:36 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec 20 15:36 drafts
-rw-rw-rw- 1 researcher2 research_team   46 Dec 20 15:36 project_k.txt
-rw-r----- 1 researcher2 research_team   46 Dec 20 15:36 project_m.txt
-rw-rw-r-- 1 researcher2 research_team   46 Dec 20 15:36 project_r.txt
-rw-rw-r-- 1 researcher2 research_team   46 Dec 20 15:36 project_t.txt
researcher2@3213bbc1d047:~/projects$
```

The provided screenshot exhibits two entered commands, followed by the output of the second command.

By observing the file "project_x.txt" starting with a period (.), it is evident that it is a hidden file. In this specific scenario, I made changes to the file permissions. Initially, I eliminated write permissions from both the user and group using the "u-w" notation. Subsequently, I proceeded to remove write permissions from the group using "g-w" and added read permissions to the group with "g+r".

Change directory permissions

In this screenshot researcher2 needs to have access to the `drafts` directory and its contents. This means that no one other than researcher2 should have execute permissions.

```
researcher2@5d738f0f927b:~/projects$ chmod g-x drafts
researcher2@5d738f0f927b:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 ..
-r--r----- 1 researcher2 research_team  46 Dec  2 15:27 .project_x.txt
drwx----- 2 researcher2 research_team 4096 Dec  2 15:27 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Dec  2 15:27 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Dec  2 15:27 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  2 15:27 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  2 15:27 project_t.txt
researcher2@5d738f0f927b:~/projects$
```

The provided screenshot presents two entered commands, followed by the output of the second command.

Prior to this, it had been established that the group possessed execute permissions. Consequently, I employed the "chmod" command to eliminate those permissions. As for the user "researcher2," they already had execute permissions assigned, making it unnecessary to add them again.

In essence, the modifications were made to revoke the execute permissions specifically from the group, while retaining the existing execute permissions for the user "researcher2." This adjustment aligns with the desired configuration of access permissions.

Summary

To align the permissions of files and directories in the projects directory with my organization's desired authorization level, I performed several modifications. Firstly, I utilized the "ls -la" command to examine the existing permissions for the directory. This step provided valuable insights that guided my subsequent actions.

After evaluating the permissions, I used the "chmod" command multiple times to make necessary changes. These alterations targeted both files and directories within the projects directory. Each instance of the "chmod" command allowed me to adjust the permissions according to the desired authorization level.

By employing this process, which involved checking the initial permissions and subsequently using the "chmod" command multiple times, I successfully aligned the permissions of files and directories in the projects directory with the desired authorization level specified by my organization.